The scikit-learn package

Ezequiel López-Rubio

Department of Computer Languages and Computer Science

University of Málaga, Spain

Contents

- Introduction
- Fundamentals
- Validation
- Models
- Performance measurement
- Conclusion

1. INTRODUCTION

1. Introduction

Overview

- The scikit-learn (also known as sklearn) package is one of the best-known Python libraries for machine learning
- It provides efficient implementations of a wide range of common machine learning algorithms
- The API is quite clean and uniform
  - This facilitates the use of different machine learning models
- The usual way to import the package is:

from sklearn import …

1. Introduction
General principles

- Consistency. All objects (basic or composite) share a consistent interface composed of a limited set of methods
- Inspection. Constructor parameters and parameter values determined by learning algorithms are stored and exposed as public attributes
- Non-proliferation of classes. Learning algorithms are the only objects to be represented using custom classes
  - Datasets are represented as NumPy arrays or SciPy sparse matrices

1. Introduction
General principles

- Composition. Whenever feasible, algorithms are implemented and composed from existing building blocks
- Sensible defaults. Whenever an operation requires a user-defined parameter, an appropriate default value is defined by the library, thereby giving a baseline solution for the task at hand

2. FUNDAMENTALS

2. Fundamentals

Supervised learning

- Given a query point $\mathbf{x}_j$ in an input space of dimension M, the task of supervised learning is to discover an estimator $y_j=h(\mathbf{x}_j)$ of an unknown function g from a set of examples $\{(\mathbf{x}_1,y_1),\ldots, (\mathbf{x}_j,y_j), \ldots, (\mathbf{x}_N,y_N)\}$ of size N. Let R be the range of g:
  - If R is a continuous set (for example, an interval of real numbers), then we have a regression problem
  - If R is a small set of discrete values, then we have a classification problem.

2. Fundamentals

Classes and hypotheses

- Each element of R is named a class
  - If the number of classes is V=2, we say that it is a binary classification problem. Otherwise, it is multiclass
  - We must choose an estimator function h (also called hypothesis) from a hypothesis space
  - An estimator performs well if it generalizes well, i.e., if it correctly predicts the value of y for novel examples
  - It is also desired that the estimators are as simple as possible. This principle is called Occam's razor

2. Fundamentals

Probabilistic classifiers and class scores

- Often, we want to design a classifier that outputs the probabilities that a given sample $\mathbf{x}_j$ belongs to each of the classes y, $P(y \mid \mathbf{x}_j)$
  - This kind of classifiers are called probabilistic
- In these cases, the estimated class $\hat{y}_j \in R$ is defined as the most likely one:

$$\hat{y}_j = \underset{y \in R}{\operatorname{argmax}}\, P(y|\mathbf{x}_j)$$

- Other classifiers output a class score for each class, i.e., the higher the score of class y, the more likely $\mathbf{x}_j$ belongs to class y
  - Usually, the class scores are not normalized and may be negative

2. Fundamentals

Data representation

- A features matrix is represented as a two-dimensional array
- Each row represents a sample, i.e., an individual
- Each column represents a feature, which may be quantitative or qualitative
- Therefore, the shape of the data table is (N, M)
- Typically, the data table is stored as an ndarray of the numpy package, or a DataFrame of the pandas package
  - Some models accept sparse matrices of the scipy package

2. Fundamentals

Data representation

- For supervised learning, a label or target array is also required, which is usually one-dimensional with shape (*N*,)
- Sometimes multiple target values are allowed so that the target array has shape (*N*, n_targets)
- The target array contains the output feature that we wish to predict from the features matrix
- The target array may contain numerical values (for regression problems) or discrete labels (for classification problems)

2. Fundamentals

Basic workflow

- Select a model class by importing from sklearn
- Instantiate the class by providing model hyperparameters to the class constructor
- Load and prepare the data as described before
- Fit the model to the data by executing the fit() method of the instance
- Test the model with new, unseen data:
  - For supervised learning, their labels can be estimated with the predict() method
  - For unsupervised learning, the transform() and predict() methods may be available

2. Fundamentals

Linear regression

- The hypothesis space for linear regression is the set of all linear functions $h_{\mathbf{w}}$ with M variables
  - For M =1 they are straight lines, for M =2 they are planes, and for M >2 they are hyperplanes

$$h_{\mathbf{w}}(\boldsymbol{x}_j) = w_0 + w_1 x_{j,1} + \ldots + w_M x_{j,M} = w_0 + \sum_{i=1}^{M} w_i x_{j,i}$$

- The w_0 term (the intercept) can be integrated into the sum by considering a dummy input x_0 whose value is always 1:

$$h_{\mathbf{w}}(\boldsymbol{x}_j) = \boldsymbol{w} \cdot \boldsymbol{x}_j = \boldsymbol{w}^T \boldsymbol{x}_j = \sum_{i=0}^{M} w_i x_{j,i}$$

2. Fundamentals

Linear regression

- Given an input $\mathbf{x}_j$ and its desired output y_j, let $\hat{y}_j$ be the predicted value, $\hat{y}_j = h(\boldsymbol{x}_j)$
- The loss $L(y_j, \hat{y}_j)$ is the utility which is lost due to the prediction $\hat{y}_j$
- For regression, a very convenient loss function is the mean squared error (MSE, also called L2 loss, or residual sum of squares):

$$L_2 = \frac{1}{N}\sum_{j=1}^{N}(y_j - \hat{y}_j)^2$$

- The LinearRegression class minimizes the MSE computed over the entire training set
  - The minimization is carried out without the need to compute any gradients or any iterative algorithm

2. Fundamentals

Linear regression

- Usually, the goal is to minimize the loss while keeping the hypothesis h simple. Then the cost to be minimized may be the regularized loss, i.e., a weighted sum of the loss L and the complexity measure, also called regularization term:

$$Cost(h_{\boldsymbol{w}}) = L(h_{\boldsymbol{w}}) + \alpha \cdot Complexity(h_{\boldsymbol{w}})$$

$$Complexity(h_{\boldsymbol{w}}) = \frac{1}{M+1}\sum_{i=0}^{M} |w_i|^q$$

$$h^* = argmin_{h_{\boldsymbol{w}}}\, Cost(h_{\boldsymbol{w}})$$

where α is a positive real number (regularization strength) and q is a positive integer (Lq regularization, q=1 or q=2).

2. Fundamentals

Linear regression

- In order to minimize the cost function, we may use the SGDRegressor class which implements the Stochastic Gradient Descent (SGD) iterative method:

$$w_i \leftarrow w_i - \eta \frac{\partial}{\partial w_i} Cost(h_{\boldsymbol{w}})$$

where η is a positive real number called the learning rate, and the partial derivative is evaluated on a random subset (batch) of the training set.

2. Fundamentals

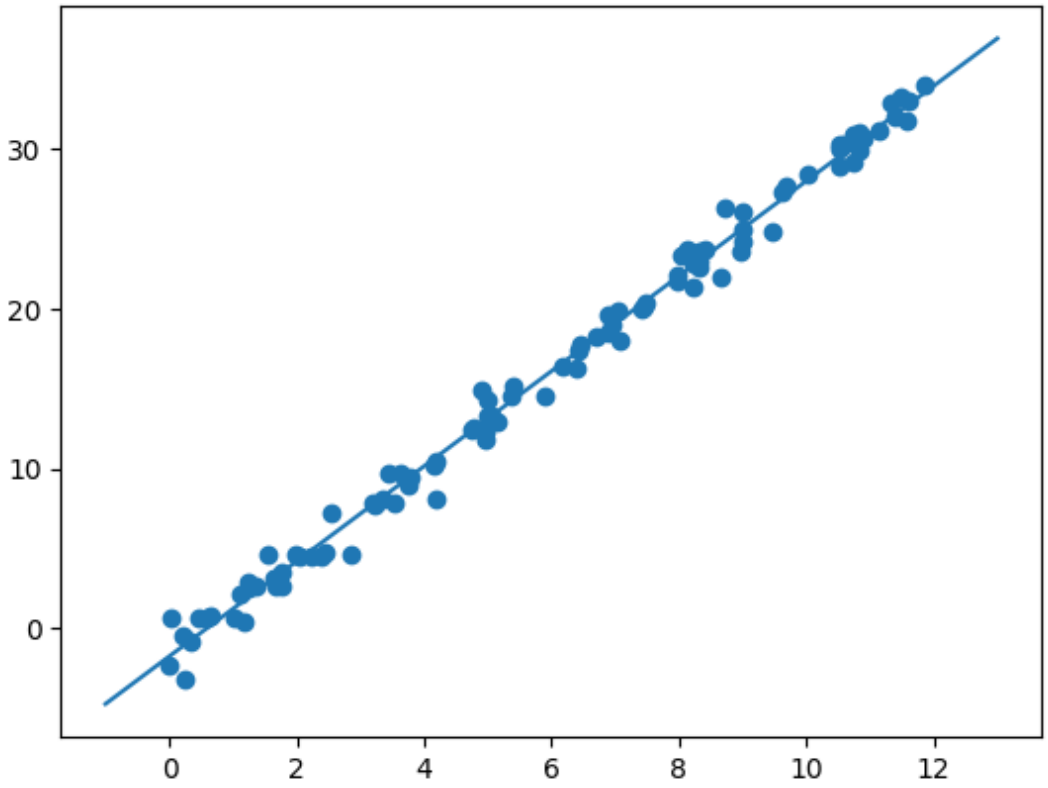
Regression example

```
import matplotlib.pyplot as plt
%matplotlib inline
import numpy as np
rng = np.random.RandomState(1)      # Fix the pseudorandom number generator seed
x = 12 * rng.rand(100)      # Generate some training samples for the x variable
y = 3 * x - 2 + rng.randn(100)     # Corresponding values for the y variable, with noise
from sklearn.linear_model import LinearRegression
model = LinearRegression(fit_intercept=True)   # Instantiate the linear regression model
X = x[:, np.newaxis]     # Convert x into a 2D array of shape (100, 1)
model.fit(X, y)             # Fit the model to the data
xfit = np.linspace(-1, 13)        # Generate test values for the x variable
Xfit = xfit[:, np.newaxis]        # Convert xfit into a 2D array
yfit = model.predict(Xfit)        # Estimate the values of the y variable for the test values of x
plt.scatter(x, y)                 # Plot the training data
plt.plot(xfit, yfit);             # Plot the estimated test values
```

2. Fundamentals

Regression example

- A linear function is estimated which minimizes the Mean Squared Error (MSE) with respect to the training samples
- The linear function does not necessarily pass through the coordinate origin because fit_intercept=True



2. Fundamentals

Naïve Bayes classifiers

- Naïve Bayes classifiers are a kind of probabilistic classifiers which are based on Bayes' theorem
  - They assume that the input features x_i are independent given the class y:

$$P(y|\mathbf{x}) = \frac{P(y)P(\mathbf{x}|y)}{\sum_{v=1}^{V} P(v)P(\mathbf{x}|v)}$$

$$P(\mathbf{x}|y) = \prod_{i=1}^{M} P(x_i|y)$$

2. Fundamentals
Naïve Bayes classifiers

- Two standard versions of the naïve Bayes classifier exist:
  - For continuous features, each feature x_i is assumed to follow a Gaussian distribution given the class y (GaussianNB class):

$$P(x_i|y) = \left(2\pi\sigma_{i,y}^2\right)^{-\frac{1}{2}} exp\left(-\frac{\left(x_i - \mu_{i,y}\right)^2}{2\sigma_{i,y}^2}\right)$$

  - For qualitative features, the probability that the k-th possible value of the i-th feature x_i occurs given the class y is estimated, $P\left(x_{i,k}|y\right)$ (MultinomialNB class)

2. Fundamentals

Classification example

```
from sklearn import datasets
iris = datasets.load_iris()              # Load the Iris standard dataset
X = iris.data                            # Set X to the features matrix
y = iris.target                          # Set y to the target array
from sklearn.model_selection import train_test_split
Xtrain, Xtest, ytrain, ytest = train_test_split(X, y, random_state=1)
from sklearn.naive_bayes import GaussianNB   # Import the Naïve Bayes model class
model = GaussianNB()                     # Instantiate the model, no hyperparameters
model.fit(Xtrain, ytrain)                # Fit the model to the training data
y_model = model.predict(Xtest)           # Predict on new test data
from sklearn.metrics import accuracy_score
accuracy_score(ytest, y_model)      # Measure the classification performance

# Output (test accuracy): 0.9736842105263158
```

2. Fundamentals
Clustering

- Determining the intrinsic grouping in a set of unlabeled data
  - Samples in the same group (cluster) are similar
  - Samples in different groups (clusters) are dissimilar
- Cluster
  - Collection of similar samples
- Algorithms
  - Partitioning methods: K-means
  - Agglomerative methods: Hierarchical clustering

2. Fundamentals

Clustering

- The *K*-means algorithm (KMeans class) partitions *N* samples $\boldsymbol{x}_j$ into *K* clusters
  - The means μ_v are the centroids of the clusters
  - It aims to minimize the Mean Squared Error (MSE, lower is better):

$$MSE_{Clustering} = \frac{1}{N}\sum_{j=1}^{N}\min_{v\in\{1,\ldots,K\}}\left\|\boldsymbol{x}_j - \mu_v\right\|^2$$

2. Fundamentals
Clustering

- Initialization
  - Set the initial *K* means
- Assignment
  - Each sample is assigned to the nearest mean
- Update
  - Calculate the new means of each cluster
- Repeat until changes below tolerance or maximum number of iterations reached

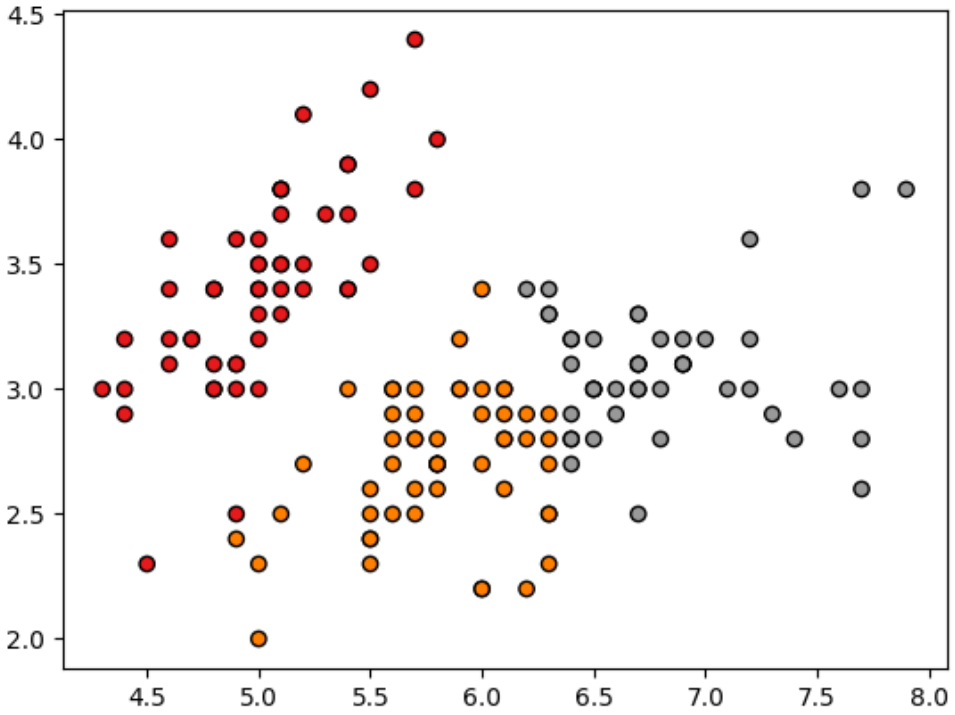
2. Fundamentals
Clustering example

```
import matplotlib.pyplot as plt
%matplotlib inline
from sklearn import datasets
iris = datasets.load_iris()                # Load the Iris standard dataset
X = iris.data[:, :2]                       # Set X to the two first features
from sklearn.cluster import KMeans   # Import the k-means model class
model = KMeans(n_clusters=3, n_init="auto")       # Instantiate the model
model.fit(X)                          # Fit the model to the training data
y = model.predict(X)              # Predict the cluster labels
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Set1, edgecolor="k");
```

2. Fundamentals

Clustering example

- Three clusters (n_clusters=3) are discovered
- Each cluster is compact, and separate from other clusters
- No class label information is provided to the clustering algorithm, i.e. this is an unsupervised learning model



3. PERFORMANCE MEASUREMENT

3. Performance evaluation
General classification performance measures

- The accuracy is the number of correctly predicted examples divided by the number of examples
- The top-*k* accuracy is the number of examples such that the correct class is among those with the *k* highest class scores (or class probabilities), divided by the number of examples
- The Rand index is the number of pairs of examples which are correctly predicted to belong to the same class, plus the number of pairs of examples which are correctly predicted to belong to different classes, divided by the number of pairs of examples
- These measures lie in the interval [0,1], and higher is better (1 means perfect classification)

3. Performance evaluation

Confusion matrix

- The confusion matrix has size $V \times V$, and its (a,b) element is the number n_{ab} of examples of class b which are predicted to belong to class a
  - The performance for the b-th class can be assessed by studying the b-th column of the matrix

	Actual class			
Predicted class		1	…	V
	1	n_{11}	…	n_{1V}
	…	…		…
	V	n_{V1}	…	n_{VV}

3. Performance evaluation

Binary classification measures

- For a binary classification problem where class 1 is called the positive class, and class 2 is called the negative class, we define:
  - True positives (TP)=n_{11}
  - True negatives (TN)=n_{22}
  - False positives (FP, also called Type I errors)=n_{12}
  - False negatives (FN, also called Type II errors)=n_{21}
- Note that:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

3. Performance evaluation

Binary classification measures

- From the preceding we define: precision (also called positive predictive value, higher is better), fallout (or false positive rate, lower is better), recall (also called sensitivity or true positive rate, higher is better), and F-measure (higher is better)

$$Precision = \frac{TP}{TP + FP}$$

$$Fallout = \frac{FP}{FP + TN}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F - measure = 2\frac{Precision \cdot Recall}{Precision + Recall}$$

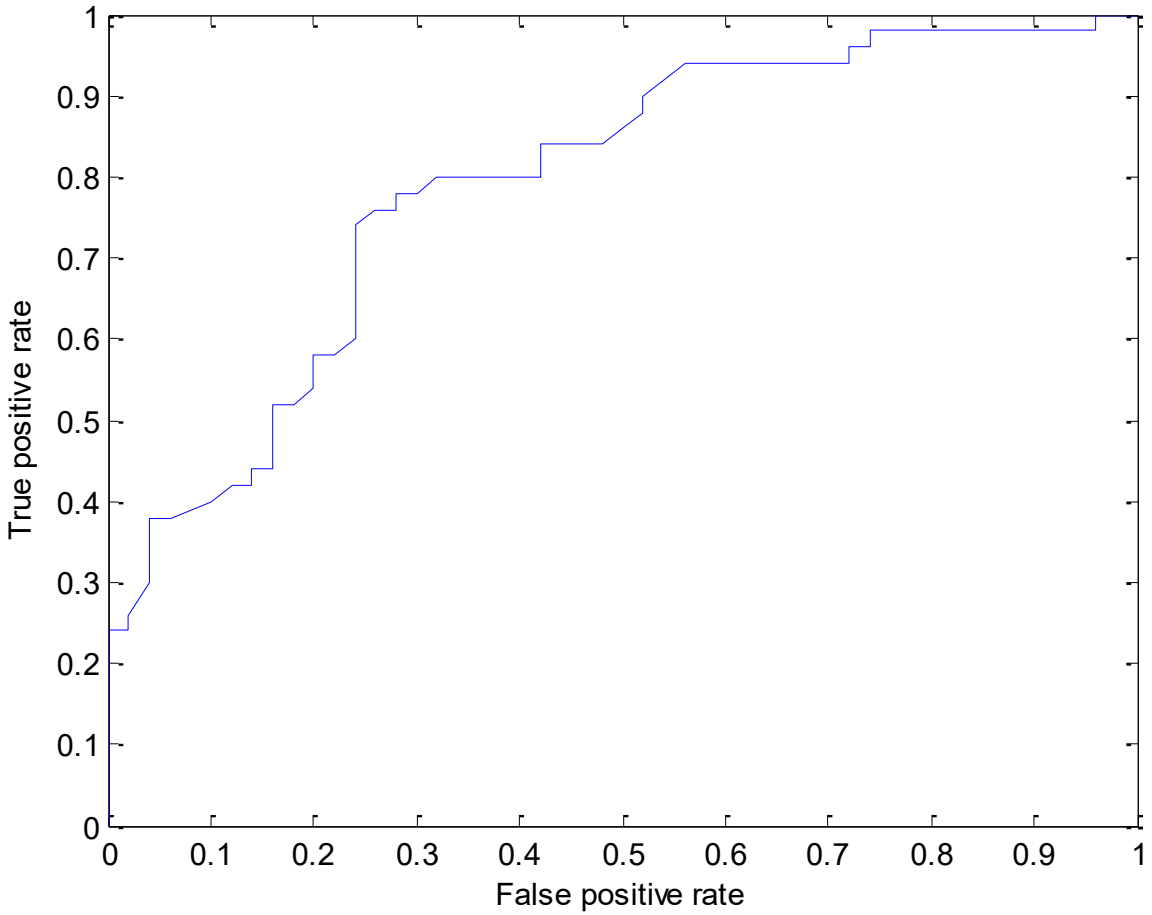
3. Performance evaluation

Receiver operating characteristic (ROC)

- For binary classification only. It is the plot of the true positive rate versus the false positive rate, as some discrimination parameter is varied
- The higher the area under the ROC curve, the better; it is a number in the interval [0,1]
- The curve illustrates the trade-off between the ratios of false positives and false negatives

3. Performance evaluation

Example of ROC curve



3. Performance measures
Classification performance in sklearn

- Performance measures can be imported from the sklearn.metrics module
- Accuracy: accuracy_score()
- Top-k accuracy: top_k_accuracy_score()
- F1 measure: f1_score()
- Precision, recall: precision_score(), recall_score()
- Receiver Operating Characteristic (ROC): roc_auc_score()
- Plot ROC curve: RocCurveDisplay class
- Confusion matrix: confusion_matrix()
- Plot confusion matrix: ConfusionMatrixDisplay class

3. Performance measures

Regression performance

- The following regression performance measures are defined: Mean squared error (MSE), lower is better; Mean absolute error (MAE), lower is better; Median absolute error (MedAE), lower is better; explained variance (EV), higher is better, 1 means perfect regression; and R2 (coefficient of determination) score, higher is better, 1 means perfect regression

$$MSE = \frac{1}{N}\sum_{j=1}^{N}(y_j - \hat{y}_j)^2$$

$$MAE = \frac{1}{N}\sum_{j=1}^{N}|y_j - \hat{y}_j|$$

$$MedAE = median(|y_j - \hat{y}_j|)$$

$$\text{EV} = 1 - \frac{var(y-\hat{y})}{var(y)}$$

$$R^2 = 1 - \frac{\sum_{j=1}^{N}(y_j - \hat{y}_j)^2}{\sum_{j=1}^{N}\left(y_j - \text{mean}(y)\right)^2}$$

3. Performance measures

Regression performance in sklearn

- They must be imported from the sklearn.metrics module
- Mean squared error (MSE): mean_squared_error()
- Mean absolute error (MAE): mean_absolute_error()
- Median absolute error: median_absolute_error()
- Explained variance: explained_variance_score()
- R2 (coefficient of determination) score: r2_score()

3. Performance measures

Clustering performance in sklearn

- From the sklearn.metrics module
- Mutual information: mutual_info_score()
- Normalized mutual information: normalized_mutual_info_score()
- Rand score: rand_score()
- Adjusted Rand score: adjusted_rand_score()

4. VALIDATION

3. Performance evaluation

Choosing the best estimator

- In order to choose the best estimator (classifier or regressor), a validation set which is disjoint with the training set must be employed
  - Otherwise, the measured performance would be too optimistic
- Several cross-validation schemes are available for performance evaluation, which split the available dataset randomly
- Holdout is fastest but less reliable, *k*-fold cross validation is in between, and leave-one-out is the slowest but most reliable
- Evaluation must take into account:
  - Problems with imbalanced classes (some classes have much more examples than others)
  - Problems where mistakes in some classes are more important that others
  - The effect of outliers, noise and missing data

3. Performance evaluation

Choosing the best estimator

- Holdout cross-validation: the set of examples is randomly split k times into training, validation and test sets. The estimator that yields the best results over the k validation sets is chosen. We report its performance over the k test sets.
- k-fold cross-validation: we split the data into k equally sized subsets. Then k learning rounds are carried out, so that each subset serves once as validation set and once as test set. Again, we choose the best estimator according to the validation performance, and we report the test performance.
- Leave-one-out cross-validation: an extreme form of k-fold cross-validation with $k=N$

4. Validation

Holdout

- Holdout validation can be performed with:

sklearn.model_selection.train_test_split(*arrays, test_size, train_size, random_state, shuffle, stratify)

- *arrays are the lists, ndarrys, or DataFrames to split (same number of rows)
- test_size and train_size represent the proportions of the dataset to be included in the test/train splits
- random_state sets the pseudorandom seed for reproducibility
- shuffle indicates whether or not to shuffle the data before splitting
- stratify indicates whether the split is done in a stratified way



4. Validation
Cross-validation

- K-fold cross-validation can be performed with:

sklearn.model_selection.KFold(n_splits, shuffle, random_state)

sklearn.model_selection.StratifiedKFold(n_splits, shuffle, random_state)

- n_splits is the number of folds
- shuffle indicates whether or not to shuffle the data before splitting
- random_state sets the pseudorandom seed for reproducibility

(train_indices, test_indices) = split(X, y, groups)

- X is the features matrix
- y is the target array (supervised learning only)
- groups is an array of group labels for the samples



3. Performance evaluation

Model capacity

- A model defines a hypothesis space for estimating the underlying function *g*
- The capacity of the model is its ability to learn (estimate) complex functions
  - The capacity increases with the variety and flexibility of the hypotheses *h* that conform the hypothesis space, and also with the number of learnable parameters of the model
- Two difficulties may arise when learning:
  - Overfitting, i.e. the model learns irrelevant details of the training set. It happens when the model has an excessive capacity. It can be noticed because the validation error is much higher than the training error (poor generalization)
  - Underfitting, i.e. the model does not have enough capacity to learn a complex function *g*. It can be noticed because both the training and validation errors remain high

4. Validation
Validation curves

```
import numpy as np
from sklearn import datasets
wine = datasets.load_wine()    # Load standard wine dataset
X = wine.data                  # Obtain the features matrix
y = wine.target                # Obtain the target array
indices = np.arange(y.shape[0])   # Get the indices of the samples
np.random.shuffle(indices)        # Shuffle the sample indices
X, y = X[indices], y[indices]       # Shuffle the samples
from sklearn.svm import SVC       # Import the Support Vector Classifier model
from sklearn.model_selection import validation_curve
my_range = np.logspace(-7, 3, 10)    # Range of the C hyperparameter to try
train_scores, valid_scores = validation_curve(SVC(kernel="linear"), X, y,
param_name="C", param_range=my_range)
```

4. Validation
Validation curves

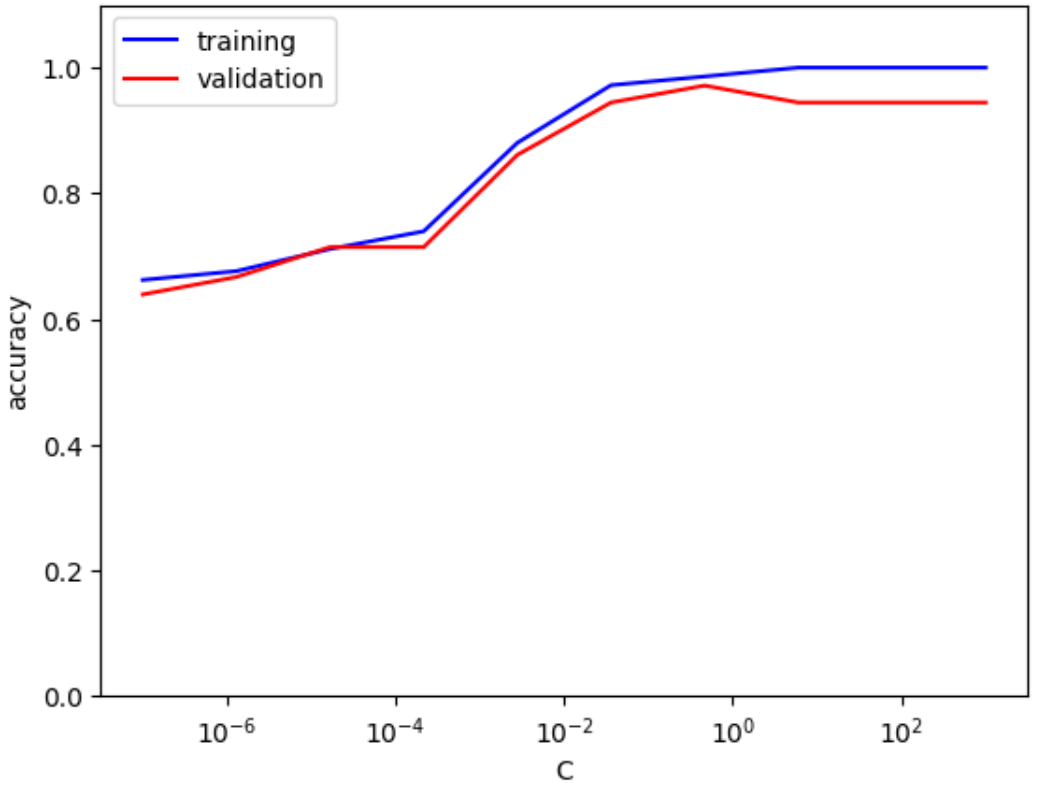
```
%matplotlib inline
import matplotlib.pyplot as plt


plt.plot(my_range, np.median(train_scores,1), color='blue', label='training')
plt.plot(my_range, np.median(valid_scores,1), color='red', label='validation')
plt.legend(loc='best')
plt.xscale('log')  # Set the x axis to log scale
plt.ylim(0, 1.1)    # Set the limits of the y axis
plt.xlabel('C')
plt.ylabel('accuracy');
```

4. Validation

Validation curves

- Best values for the *C* inverse regularization hyperparameter of the SVC classifier can be easily spotted
- The performance on the training set is usually better than the validation set
- The performance on the validation set is more realistic than the training set



4. Validation
Learning curves

```
import numpy as np
from sklearn import datasets
wine = datasets.load_wine()    # Load standard wine dataset
X = wine.data                  # Obtain the features matrix
y = wine.target                # Obtain the target array
indices = np.arange(y.shape[0])   # Get the indices of the samples
np.random.shuffle(indices)        # Shuffle the sample indices
X, y = X[indices], y[indices]       # Shuffle the samples
from sklearn.svm import SVC       # Import the Support Vector Classifier model
from sklearn.model_selection import learning_curve
my_sizes = np.linspace(0.01, 1, 25)   # Training set sizes
N, train_lc, val_lc = learning_curve(SVC(kernel="linear"), X, y, cv=10,
train_sizes=my_sizes)
```

4. Validation
Learning curves

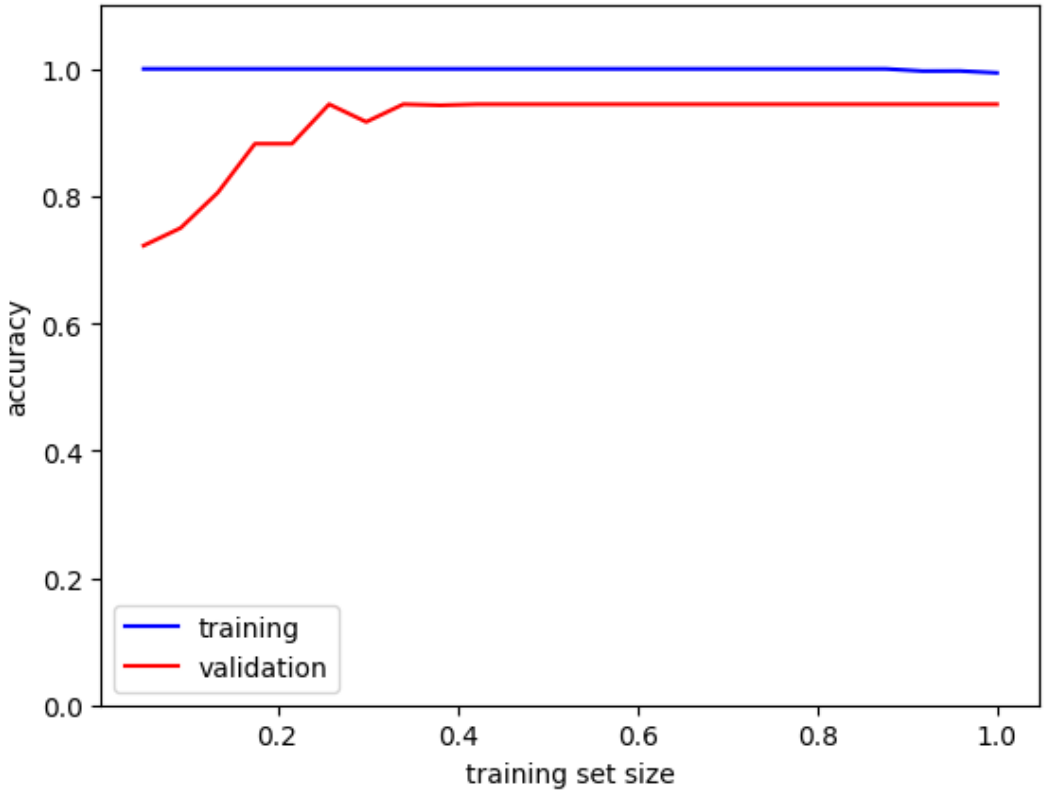
```
%matplotlib inline
import matplotlib.pyplot as plt

plt.plot(my_sizes, np.median(train_lc,1), color='blue', label='training')
plt.plot(my_sizes, np.median(val_lc,1), color='red', label='validation')
plt.legend(loc='best')
plt.ylim(0, 1.1)
plt.xlabel('training set size')
plt.ylabel('accuracy');
```

4. Validation

Learning curves

- The classification performance increases as the training set size grows, until a stable point is reached
- From that point, more training data does not produce better performance
- Other classification models with more learning capacity should be tried in order to exploit the additional training data



4. Validation
Grid search

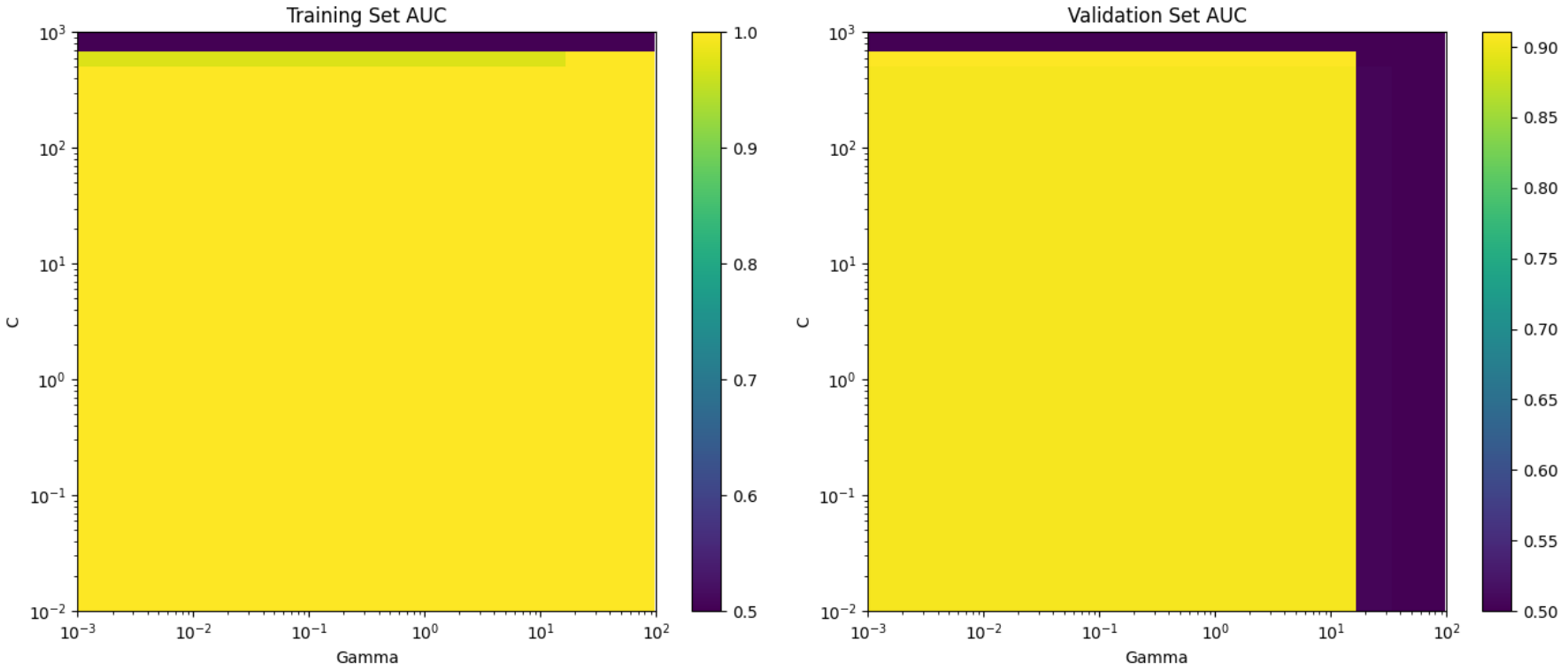
- Exhaustive grid search cross-validation:

sklearn.model_selection.GridSearchCV(estimator, param_grid, scoring, cv)

- estimator is the estimator object, assumed to implement the scikit-learn estimator interface
- param_grid is a dict with parameters names as keys and lists of parameter settings to try as values, or a list of such dicts
- scoring is the strategy to evaluate the performance of the cross-validated model on the test set
- cv is the cross-validation splitting strategy

4. Validation
Grid search

- The C and gamma hyperparameters of a SVC classifier are optimized for the Breast Cancer Wisconsin dataset, using the Area Under the Curve (AUC, higher is better) as the performance measure



5. MODELS

5. Models

Naïve Bayes classifier

- Gaussian Naïve Bayes for continuous features:

sklearn.naive_bayes.GaussianNB(priors, var_smoothing)

- priors is an array-like of shape (n_classes,) with the prior probabilities of the classes. If specified, the priors are not adjusted according to the data
- var_smoothing is the portion of the largest variance of all features that is added to variances for calculation stability

5. Models

Naïve Bayes classifier

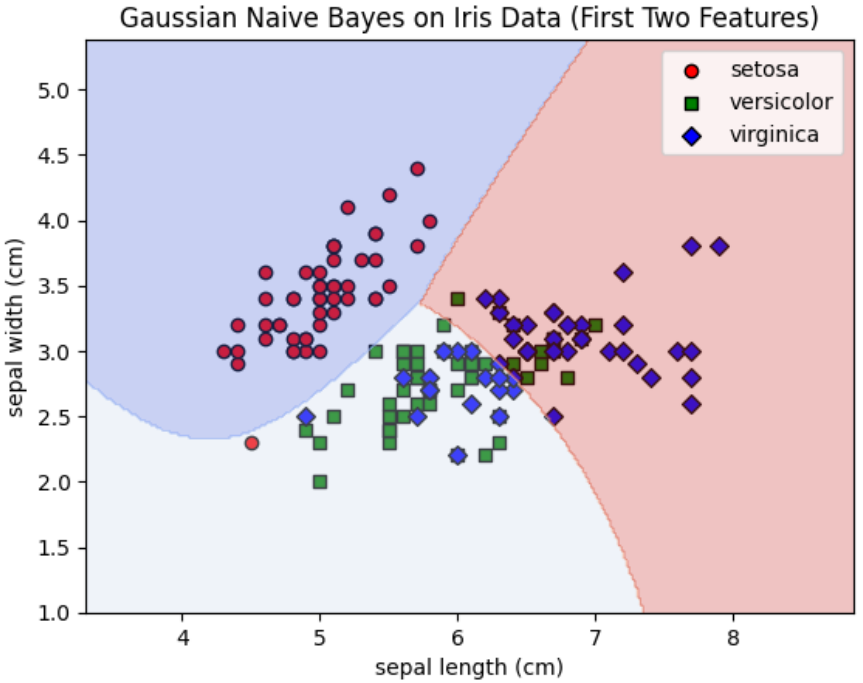
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.naive_bayes import GaussianNB

iris = load_iris() # Load the Iris dataset
X = iris.data[:, :2]  # Use only the first two features
y = iris.target
clf = GaussianNB()
clf.fit(X, y) # Train the classifier
markers = ['o', 's', 'D']  # Markers for the three classes
colors = ['red', 'green', 'blue']
for i, color, marker in zip(np.unique(y), colors, markers):
    plt.scatter(X[y == i, 0], X[y == i, 1], color=color,
label=iris.target_names[i], marker=marker,
edgecolor='k')
# Create a mesh grid for the decision boundaries
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
np.arange(y_min, y_max, 0.02))
# Predict on the mesh grid to determine decision
boundaries
Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
# Plot the decision boundaries
plt.contourf(xx, yy, Z, alpha=0.3,
cmap=plt.cm.coolwarm)
plt.xlabel(iris.feature_names[0])
plt.ylabel(iris.feature_names[1])
plt.title("Gaussian Naive Bayes on Iris Data (First Two
Features)")
plt.legend()
plt.show()
```

5. Models

Naïve Bayes classifier

- Gaussian Naïve Bayes is employed because the input features are continuous
- The Naïve Bayes classifier builds nonlinear decision boundaries for this multiclass problem
- Each point in the plane belongs to the class which has the highest posterior probability for that point, according to the Naïve Bayes probabilistic model



4. Models

Naïve Bayes classifier

- Multinomial Naïve Bayes for discrete features:

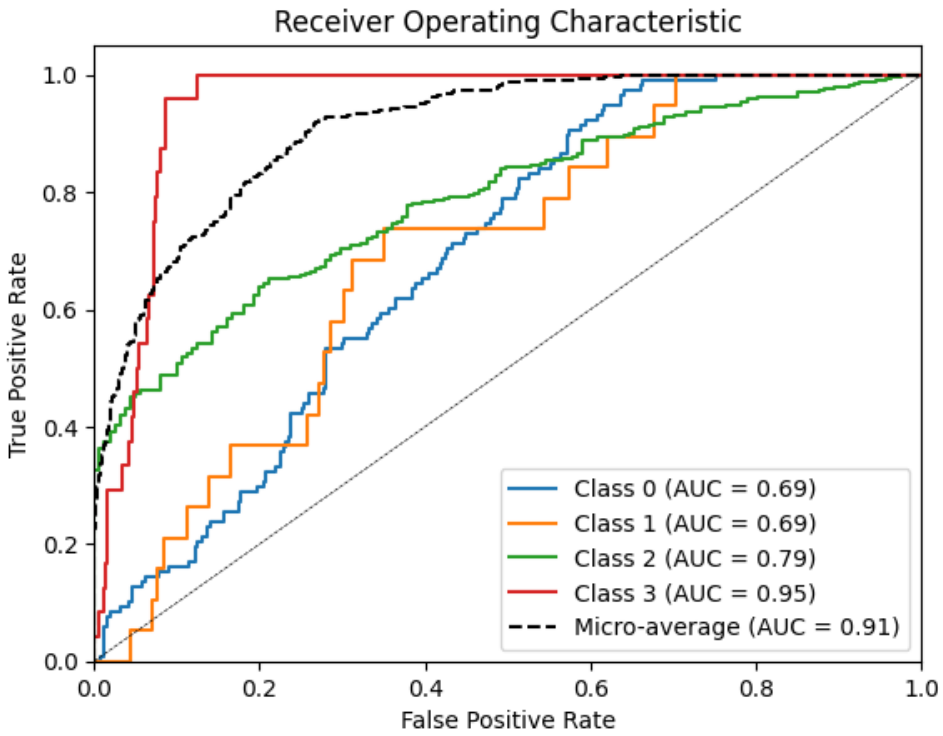
sklearn.naive_bayes.MultinomialNB(alpha, force_alpha, fit_prior, class_prior)

- alpha is an additive smoothing parameter (set alpha=0 and force_alpha=True, for no smoothing)
- force_alpha If False and alpha is less than 1e-10, it will set alpha to 1e-10. If True, alpha will remain unchanged
- fit_prior indicates whether to learn class prior probabilities or not
- class_prior are the prior probabilities of the classes. If specified, the priors are not adjusted according to the data

5. Models

Naïve Bayes classifier

- Multinomial Naïve Bayes is employed for the Car Evaluation dataset which contains discrete features
- The Receiver Operating Characteristic curve (ROC) is plotted for each class, and the associated AUC performance measure is reported
- The micro-average method is employed to obtain an overall ROC which encompasses all four classes



5. Models

Nearest neighbor classifier

- The *K*-nearest neighbors classifier assigns an input sample $\mathbf{x}_k$ to the majority class within the *K* nearest neighbors of $\mathbf{x}_k$ where the distance is computed according to some metric

sklearn.neighbors.KNeighborsClassifier(n_neighbors, p=2, metric)

- n_neighbors is the number of neighbors to use (*K*)
- p is the power parameter for the Minkowski metric (p=2 for the standard Euclidean distance)
- metric is the metric to use for distance computation

5. Models

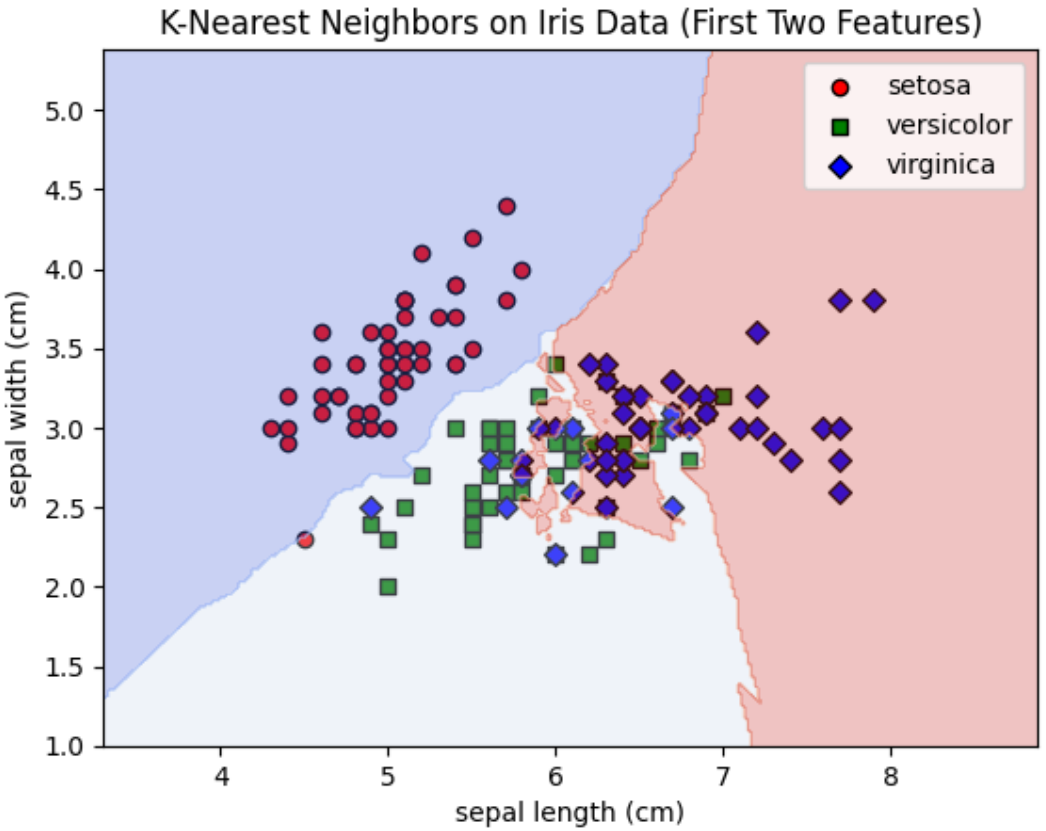
Nearest neighbor classifier

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
iris = load_iris() # Load the Iris dataset
X = iris.data[:, :2]  # Use only the first two features
y = iris.target
# Train a K-Nearest Neighbors classifier
clf = KNeighborsClassifier()
clf.fit(X, y)
markers = ['o', 's', 'D']  # Markers for the three classes
colors = ['red', 'green', 'blue']
for i, color, marker in zip(np.unique(y), colors,
markers):
    plt.scatter(X[y == i, 0], X[y == i, 1], color=color,
label=iris.target_names[i], marker=marker,
edgecolor='k')
# Create a mesh grid for the decision boundaries
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
np.arange(y_min, y_max, 0.02))
# Predict on the mesh grid to determine decision
boundaries
Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
# Plot the decision boundaries
plt.contourf(xx, yy, Z, alpha=0.3,
cmap=plt.cm.coolwarm)
plt.xlabel(iris.feature_names[0])
plt.ylabel(iris.feature_names[1])
plt.title("K-Nearest Neighbors on Iris Data (First Two
Features)")
plt.legend()
plt.show()
```

5. Models

Nearest neighbors classifier

- Each point in the plane belongs to the class which has a majority of nearest neighbors for that point
- The nearest neighbor classifier builds ragged decision boundaries, which indicates overfitting
- Less overfitting can be obtained by raising the n_neighbors hyperparameter



5. Models
Multilayer perceptron

- The multilayer perceptron (MLP) is a type of artificial neural network (ANN)
  - It comprises several neural layers: an input layer (*M* inputs), several hidden layers, and an output layer (*Q* outputs)
  - Each layer contains several artificial neurons
  - Each artificial neuron computes a weighted sum of its inputs, and then applies an activation function *f*, which is typically non-linear
- The L2 regularized loss is commonly employed as cost function for regression
- The cross-entropy (also called log-loss) is usually employed as cost function for classification with *V* classes:

$$L_{CE} = -\frac{1}{N}\sum_{v=1}^{V}\sum_{y_j=v} log(z_{j,v})$$

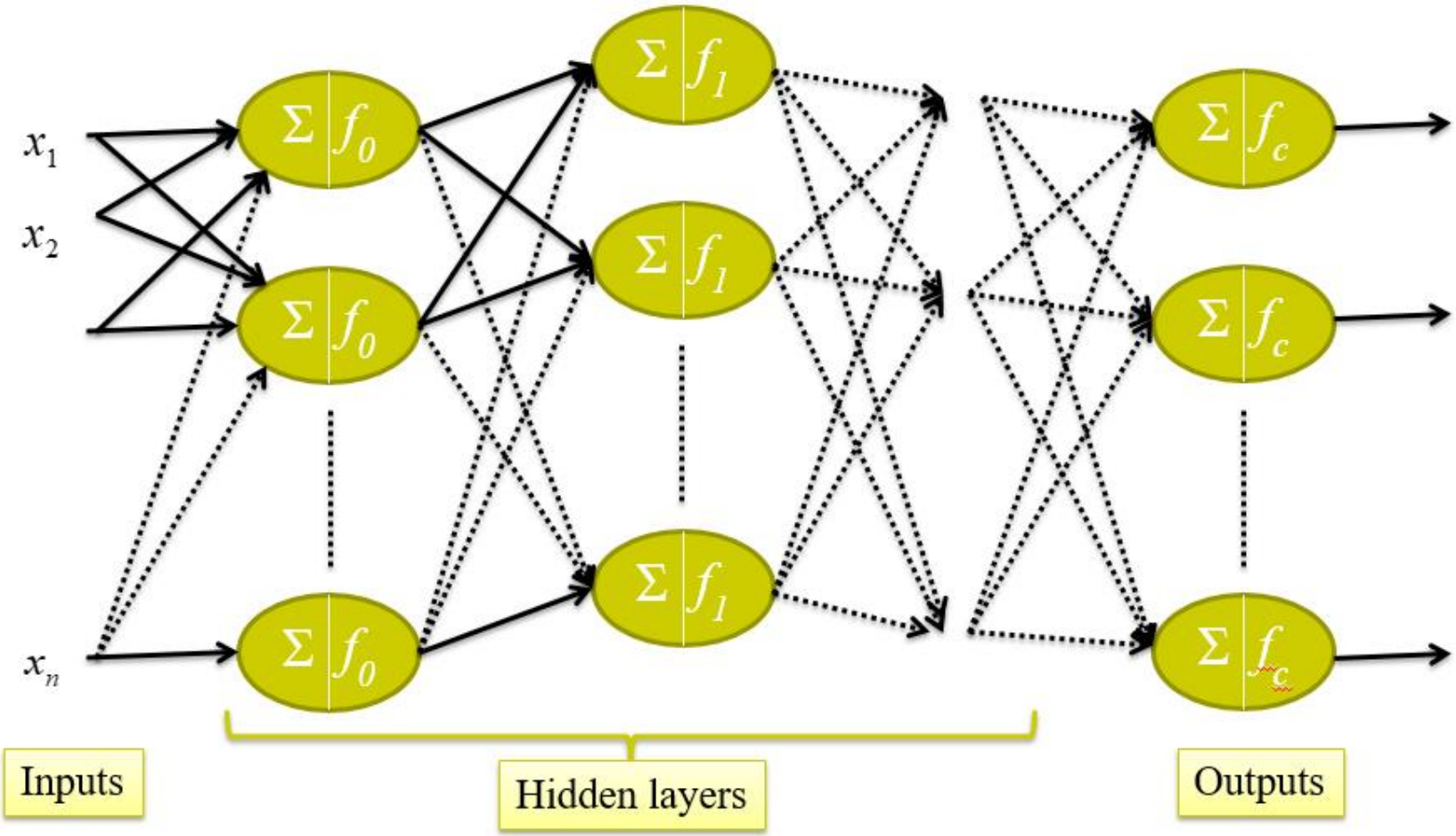
where $z_{j,v} \in (0,1)$ is the output of the v-th output neuron for input $\boldsymbol{x}_j$, $j \in \{1, \dots, N\}$.

- For V=2, binary cross-entropy (BCE) is employed:

$$L_{BCE} = -\frac{1}{N}\sum_{j=1}^{N} y_j log(z_{j,1}) + (1 - y_j) log(1 - z_{j,1})$$

5. Models

Multilayer perceptron



5. Models
Multilayer perceptron classifier/regressor

sklearn.neural_network.MLPClassifier(hidden_layer_sizes, activation, solver, alpha, batch_size, learning_rate_init, max_iter)

sklearn.neural_network.MLPRegressor(hidden_layer_sizes, activation, solver, alpha, batch_size, learning_rate_init, max_iter)

- hidden_layer_sizes represents the number of neurons in the hidden layers
- activation is the activation function *f* for the hidden layers
- solver is the solver for weight optimization
- alpha is the strength of the L2 regularization term
- batch_size is the size of minibatches for stochastic optimizers
- learning_rate_init is the initial learning rate used
- max_iter is the maximum number of iterations

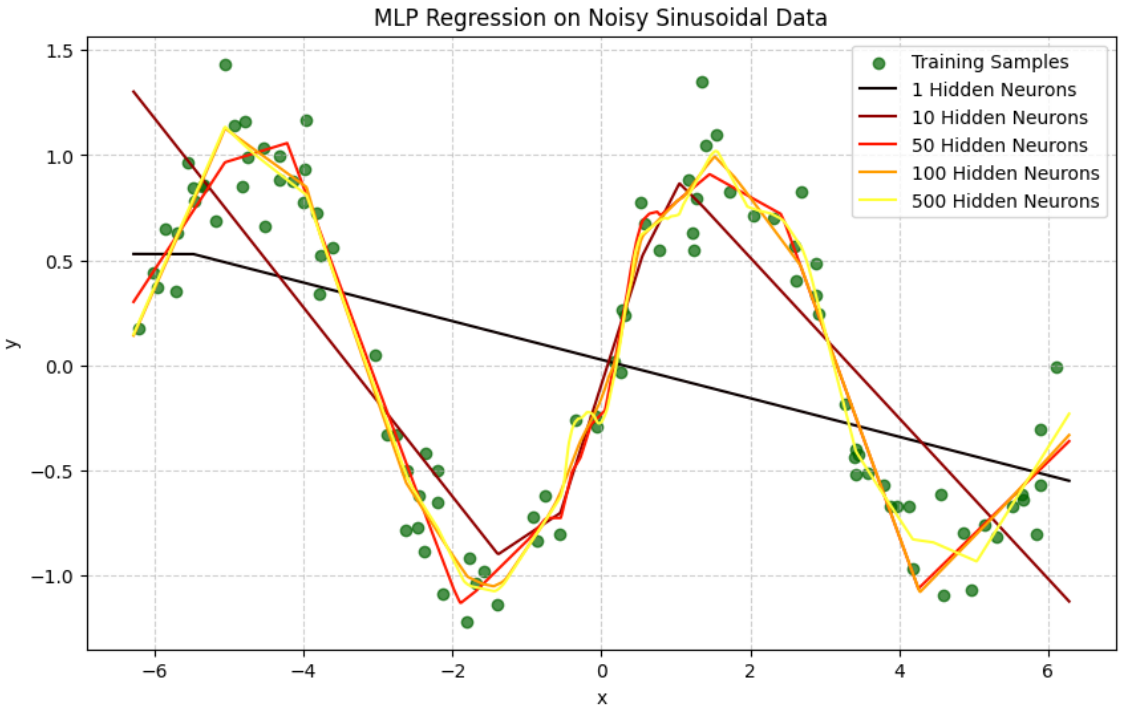
5. Models
Multilayer perceptron regressor

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPRegressor
from matplotlib.cm import get_cmap

num_samples = 100
x_range = (-2 * np.pi, 2 * np.pi)
noise_std = 0.2
np.random.seed(42)  # For reproducibility
x_train = np.random.uniform(x_range[0], x_range[1], num_samples)
y_train = np.sin(x_train) + np.random.normal(0, noise_std, num_samples)
hidden_neurons_list = [1, 10, 50, 100, 500]
# Create a dense grid of x-values for plotting learned functions
x_plot = np.linspace(x_range[0], x_range[1], 1000)
# Train MLPs and predict over x_plot
cmap = get_cmap("hot")
mlp_predictions = []
for i, hidden_neurons in enumerate(hidden_neurons_list):
    mlp = MLPRegressor(hidden_layer_sizes=(hidden_neurons,),
                       max_iter=100_000, solver='lbfgs',
                       random_state=42)
    mlp.fit(x_train.reshape(-1, 1), y_train)
    y_pred = mlp.predict(x_plot.reshape(-1, 1))
    mlp_predictions.append((hidden_neurons, y_pred))
# Plotting
plt.figure(figsize=(10, 6))
plt.scatter(x_train, y_train, color='darkgreen', label='Training Samples', alpha=0.7)
for i, (hidden_neurons, y_pred) in enumerate(mlp_predictions):
    color = cmap(i / len(hidden_neurons_list))  # Select color from "hot" colormap
    plt.plot(x_plot, y_pred, color=color, label=f'{hidden_neurons} Hidden Neurons')
plt.xlabel('x')
plt.ylabel('y')
plt.title('MLP Regression on Noisy Sinusoidal Data')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

5. Models
Multilayer perceptron regressor

- The MLP with only one hidden neuron builds a very crude approximation to the underlying sine function y=sin(x), which indicates underfitting
- Adding more hidden neurons increases the learning capacity of the MLP, which eliminates underfitting
- Tuning the learning hyperparameters is critical to obtain a good fit



5. Models

Linear regression

- Ordinary least squares linear regression:

sklearn.linear_model.LinearRegression(fit_intercept)

- fit_intercept indicates whether to calculate the intercept for this model. If set to False, no intercept will be used in calculations (i.e., data is expected to be centered)
- Stochastic gradient descent linear regression:

sklearn.linear_model.SGDRegressor(loss, penalty, alpha, fit_intercept, max_iter, learning_rate)

- loss is the loss function, penalty is the regularization term, alpha controls the regularization

5. Models

Logistic regression

- In spite of its name, it is a classification model
- For binary classification, each hypothesis contains a logistic function that yields a number between 0 and 1 that represents the probability that sample $\boldsymbol{x}_j$ belongs to the positive class C_1:

$$h_{\boldsymbol{w}}(\boldsymbol{x}_j) = P(\boldsymbol{x}_j \in C_1) =$$

$$Logistic\left(-\sum_{i=0}^{M} w_i x_{j,i}\right) =$$

$$Logistic(-\boldsymbol{w}^T \boldsymbol{x}_j) = \frac{1}{1 + exp(-\boldsymbol{w}^T \boldsymbol{x}_j)}$$

- The cost function is the regularized binary cross entropy:

$$L_{BLR} = L_{BCE} + \frac{\alpha}{M+1} \sum_{i=0}^{M} |w_i|^q$$

5. Models

Logistic regression

- For multiclass classification, each hypothesis contains V exponential functions, each with its associated weight vector $\boldsymbol{w}_r$, that are normalized by the softmax function to yield a number between 0 and 1 that represents the probability that sample $\boldsymbol{x}_j$ belongs to the associated class S_r:

$$P(\boldsymbol{x}_j \in S_r) = \frac{exp(\boldsymbol{w}_r{}^T \boldsymbol{x}_j)}{\sum_{v=1}^{V} exp(\boldsymbol{w}_v{}^T \boldsymbol{x}_j)}$$

- The cost function is the regularized cross entropy:

$$L_{LR} = L_{CE} + \frac{\alpha}{V(M+1)} \sum_{v=1}^{V} \sum_{i=0}^{M} |w_{v,i}|^q$$

5. Models

Logistic regression classifier

- Logistic regression classifier:

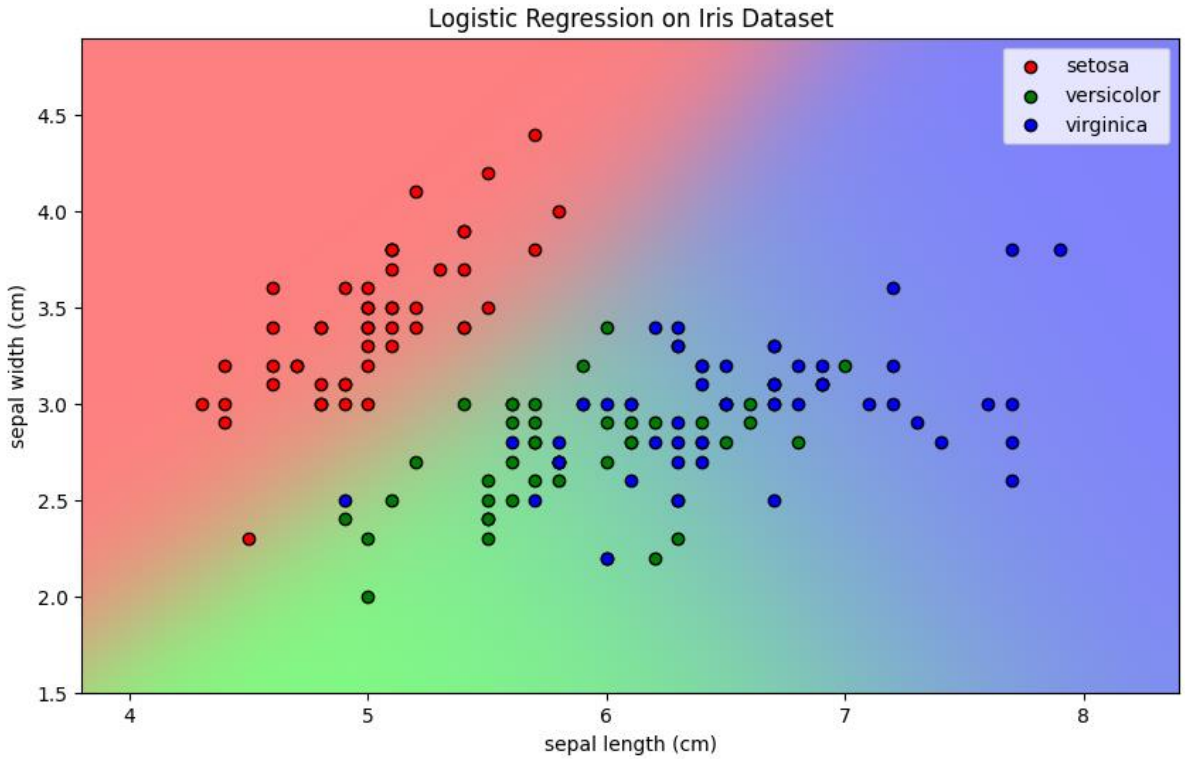
sklearn.linear_model.LogisticRegression(penalty, C, fit_intercept, solver, max_iter)

- penalty specifies the norm of the penalty (q)
- C is the inverse of the regularization strength α. The strength of the regularization is inversely proportional to C. Must be strictly positive
- fit_intercept indicates whether to calculate the intercept (w_0) for this model. If set to False, no intercept will be used in calculations (i.e., data is expected to be centered)
- solver is the algorithm to solve the optimization problem
- max_iter is the maximum number of iterations

5. Models

Logistic regression classifier

- The logistic regression classifier builds very simple linear decision boundaries, i.e. the model is underfitted
- Each point in the plane has a RGB color proportional to the three class probabilities assigned by the model



5. Models

Linear Discriminant Analysis

- Let $P(r)$ be the prior probability of each class $r \in V$. Bayes' theorem gives the posterior probability of class r, given the observed data $\mathbf{x}$:

$$P(r|\mathbf{x}) = \frac{P(r)P(\mathbf{x}|r)}{\sum_{v=1}^{V} P(v)P(\mathbf{x}|v)}$$

- Let us assume that the probability distribution of each class is multivariate Gaussian:

$$p(\mathbf{x}|v) = \frac{1}{(2\pi)^{\frac{M}{2}}\left(\det(\mathbf{C}_v)\right)^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_v)^T \mathbf{C}_v{}^{-1}(\mathbf{x}-\boldsymbol{\mu}_v)\right)$$

5. Models
Linear Discriminant Analysis

- A test sample $\mathbf{x}$ is estimated to belong to the class r with the highest probability $P(r|\mathbf{x})$. This is equivalent to choose the class with the maximum class score:

$$r = \arg\max_v \left(2\log\left(P(v)\right) - \log\left(\det(\mathbf{C}_v)\right) - (\mathbf{x} - \boldsymbol{\mu}_v)^T \mathbf{C}_v{}^{-1} (\mathbf{x} - \boldsymbol{\mu}_v)\right)$$

- Two models are available:
  - Linear Discriminant Analysis (LDA) assumes that there is a common covariance matrix $\mathbf{C}$ for all classes, which is the average of the covariance matrices of all classes
  - Quadratic Discriminant Analysis (QDA) employs a different covariance matrix $\mathbf{C}_v$ for each class v

5. Models
Linear Discriminant Analysis

- Linear Discriminant Analysis (LDA) classifier:

sklearn.discriminant_analysis.LinearDiscriminantAnalysis(solver, priors)

- solver is the algorithm to solve the optimization problem of finding the per-class mean vectors $\boldsymbol{\mu}_v$ and the global covariance matrix $\mathbf{C}$
- priors indicates the class prior probabilities $P(v)$

5. Models
Linear Discriminant Analysis

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

# Load the Iris dataset
iris = load_iris()
X = iris.data[:, :2]  # Use only the first two features
y = iris.target

# Train a Linear Discriminant Analysis classifier
clf = LinearDiscriminantAnalysis()
clf.fit(X, y)

# Plot the training data with each class in a different color and marker
markers = ['o', 's', 'D']  # Markers for the three classes
colors = ['red', 'green', 'blue']  # Colors for the three classes

for i, color, marker in zip(np.unique(y), colors, markers):
    plt.scatter(X[y == i, 0], X[y == i, 1], color=color,
label=iris.target_names[i], marker=marker, edgecolor='k')

# Create a mesh grid for the decision boundaries
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
np.arange(y_min, y_max, 0.02))

# Predict on the mesh grid to determine decision boundaries
Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

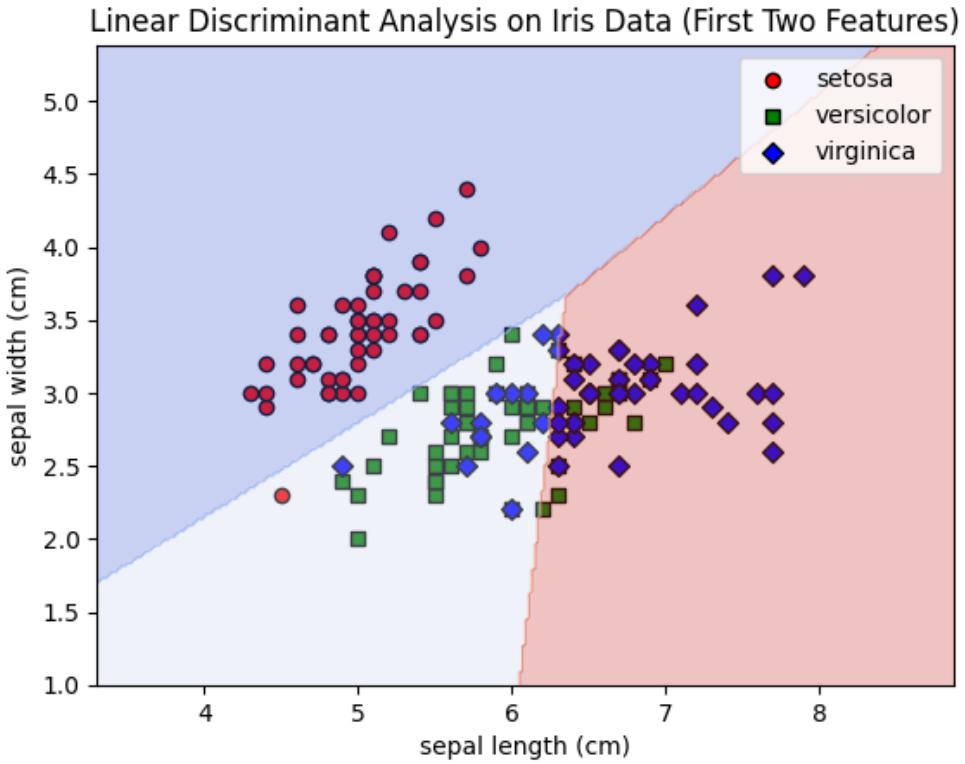
# Plot the decision boundaries
plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)

plt.xlabel(iris.feature_names[0])
plt.ylabel(iris.feature_names[1])
plt.title("Linear Discriminant Analysis on Iris Data (First Two Features)")
plt.legend()
plt.show()
```

5. Models

Linear Discriminant Analysis

- The LDA classifier builds linear decision boundaries
- These decision boundaries are too simple, i.e. there is underfitting
- Each point in the plane belongs to the class which has the highest posterior probability for that point according to the LDA multivariate Gaussian probabilistic model



5. Models

Support vector classification

- For binary classification, each hypothesis yields -1 or 1 indicating that sample $\boldsymbol{x}_j$ belongs to the negative or the positive class, respectively:

$$h_{\boldsymbol{w}}(\boldsymbol{x}_j) = sign\left(\boldsymbol{w}^T \phi(\boldsymbol{x}_j)\right)$$

where ϕ is a transformation function

- The following cost function must be minimized:

$$L_{SVC} = \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} + C\sum_{j=1}^{N} \zeta_j$$

where C is the inverse regularization strength, and ζ_j is the distance between $\boldsymbol{x}_j$ and the margin boundary of its class:

$$y_j\left(\boldsymbol{w}^T \phi(\boldsymbol{x}_j)\right) \geq 1 - \zeta_j$$
$$\zeta_j \geq 0$$

where $y_j \in \{-1,1\}$.

5. Models

Support vector classification

- The kernel function K is specified rather than the transformation function ϕ :

$$K(\boldsymbol{x}_s, \boldsymbol{x}_t) = \phi(\boldsymbol{x}_s)^T \phi(\boldsymbol{x}_t)$$

There are several kernels available:

Linear: $K(\boldsymbol{x}_s, \boldsymbol{x}_t) = \boldsymbol{x}_s{}^T \boldsymbol{x}_t$

Polynomial: $K(\boldsymbol{x}_s, \boldsymbol{x}_t) = (\gamma \boldsymbol{x}_s{}^T \boldsymbol{x}_t + \theta)^\beta$

Radial basis function (RBF): $K(\boldsymbol{x}_s, \boldsymbol{x}_t) = exp(-\gamma \|\boldsymbol{x}_s - \boldsymbol{x}_t\|^2)$

Hyperbolic tangent: $K(\boldsymbol{x}_s, \boldsymbol{x}_t) = tanh(\boldsymbol{x}_s{}^T \boldsymbol{x}_t + \theta)$

- The values of the hyperparameters γ, θ, β must be determined by cross-validation

5. Models

Support vector classification

- For multiclass problems, two approaches are available:
  - The "One versus Rest" (OVR) approach returns an array of shape (N, C) with the decision function values for each class versus the rest
  - The "One versus One" (OVO) approach returns an array of shape $(N, \frac{C(C-1)}{2})$ with the decision function values for each pair of classes
- Class probabilities are computed on demand
  - The probability computation slows down the training process

5. Models

Support vector classification

- Support Vector Classification:

sklearn.svm.SVC(C, kernel, degree, gamma, coef0, decision_function_shape, probability)

- C is the regularization parameter. The strength of the regularization is inversely proportional to C. Must be strictly positive. The penalty is a squared L2 penalty.
- kernel specifies the kernel type to be used in the algorithm
- degree is the degree β of the polynomial kernel function
- gamma is the kernel coefficient γ for 'rbf', 'poly' and 'sigmoid' kernels
- coef0 is the independent term θ for 'poly' and 'sigmoid' kernel functions
- decision_function_shape switches between 'ovo' and 'ovr' for multiclass problems
- probability indicates whether to enable probability estimates

5. Models

Support vector regression

- For regression, each hypothesis yields the estimated value of the underlying function for sample $\boldsymbol{x}_j$:

$$h_{\boldsymbol{w}}(\boldsymbol{x}_j) = \boldsymbol{w}^T \phi(\boldsymbol{x}_j)$$

- The following cost function must be minimized:

$$L_{SVR} = \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} + C\sum_{j=1}^{N}(\zeta_j + \zeta_j^*)$$

where C is the inverse regularization strength, ζ_j is the distance from the estimation to the true value y_j when the estimation $h_{\boldsymbol{w}}(\boldsymbol{x}_j)$ exceeds y_j by more than a predefined margin ε, and ζ_j^* is the same for estimations that are more than ε below y_j:

$$y_j - \boldsymbol{w}^T\phi(\boldsymbol{x}_j) \leq \varepsilon + \zeta_j$$
$$\boldsymbol{w}^T\phi(\boldsymbol{x}_j) - y_j \leq \varepsilon + \zeta_j^*$$
$$\zeta_j, \zeta_j^* \geq 0$$

5. Models

Support vector regression

- Support Vector Regression:

sklearn.svm.SVR(kernel, degree, gamma, coef0, C, epsilon)

- kernel specifies the kernel type to be used in the algorithm
- degree is the degree β of the polynomial kernel function
- gamma is the kernel coefficient γ for 'rbf', 'poly' and 'sigmoid' kernels
- coef0 is the independent term θ for 'poly' and 'sigmoid' kernel functions
- C is the regularization parameter. The strength of the regularization is inversely proportional to C. Must be strictly positive. The penalty is a squared L2 penalty
- epsilon is the margin parameter ε, must be strictly positive

5. Models

Support vector regression

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVR
from matplotlib.cm import get_cmap
from matplotlib import colormaps
num_samples = 100
x_range = (-2 * np.pi, 2 * np.pi)
noise_std = 0.2
np.random.seed(42)  # For reproducibility
x_train = np.random.uniform(x_range[0], x_range[1], num_samples)
y_train = np.sin(x_train) + np.random.normal(0, noise_std, num_samples)
c_values_list = [0.01, 0.2, 0.4, 0.6, 0.8]
x_plot = np.linspace(x_range[0], x_range[1], 1000)
# Train SVRs and predict over x_plot
cmap = colormaps["hot"]
svr_predictions = []
for i, c_value in enumerate(c_values_list):
    svr = SVR(C=c_value)  # Create SVR with specified C value
    svr.fit(x_train.reshape(-1, 1), y_train)
    y_pred = svr.predict(x_plot.reshape(-1, 1))
    svr_predictions.append((c_value, y_pred))

# Plotting
plt.figure(figsize=(10, 6))

# Plot training data
plt.scatter(x_train, y_train, color='darkgreen', label='Training Samples', alpha=0.7)

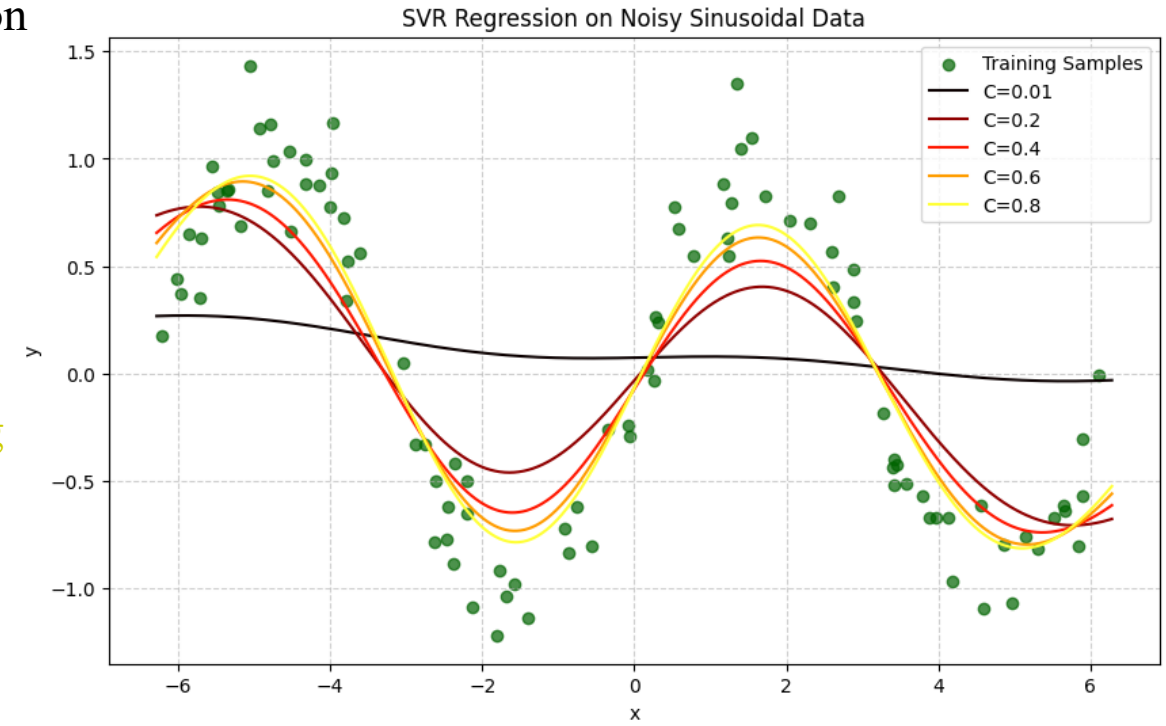
# Plot learned functions for each SVR
for i, (c_value, y_pred) in enumerate(svr_predictions):
    color = cmap(i / len(c_values_list))  # Select color from "hot" colormap
    plt.plot(x_plot, y_pred, color=color, label=f'C={c_value}')

# Plot settings
plt.xlabel('x')
plt.ylabel('y')
plt.title('SVR Regression on Noisy Sinusoidal Data')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

5. Models

Support vector regression

- The SVR model builds a smooth approximation to the underlying sine function y=sin(x)
- The inverse regularization hyperparameter C controls the underfitting/overfitting balance
- For C=0.01 the regularization is too strong, so the model is underfitted



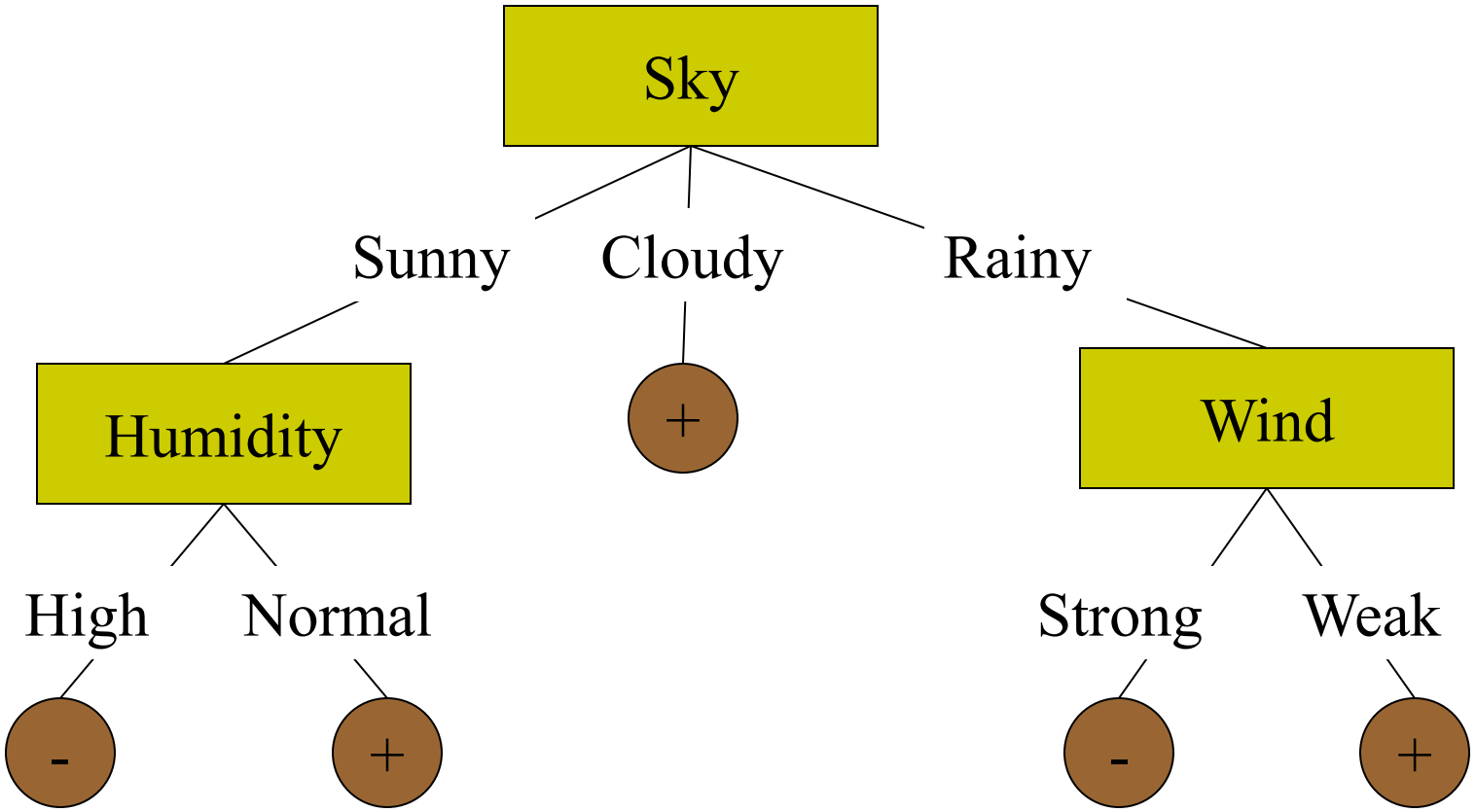
5. Models

Decision tree classifier

- A decision tree produces its decision by performing a sequence of tests
  - Each internal node tests the value of one of the input attributes x_i
  - Branches from the node are labeled with the possible values of the attribute $\lambda \in \text{Values}(x_i)$
  - Each leaf node specifies a value y to be returned by the classifier
  - Each path leading to a leaf node can be represented as a sentence (rule) of propositional logic

5. Models

Decision tree classifier



5. Models

Decision tree classifier

- Decision tree classifier:

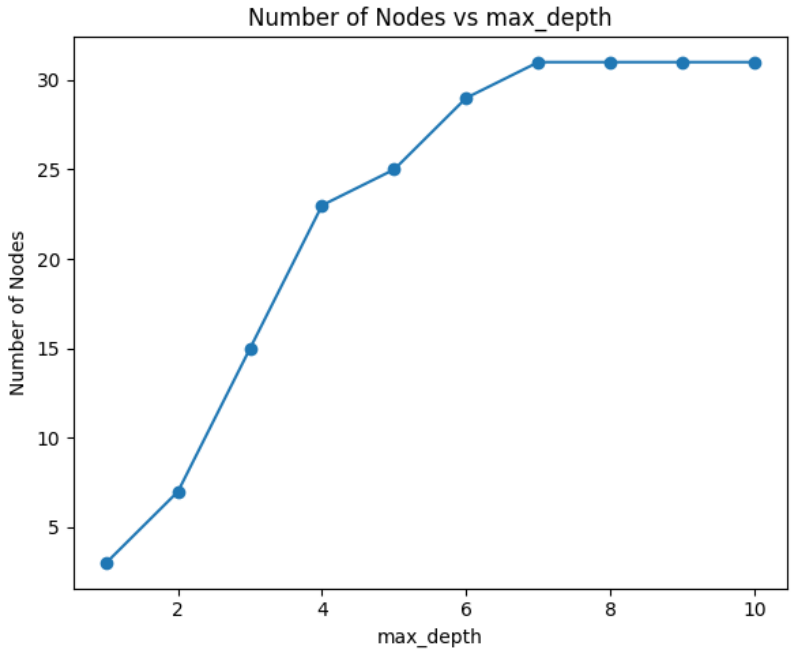
sklearn.tree.DecisionTreeClassifier(criterion, splitter, max_depth, min_samples_split, min_samples_leaf)

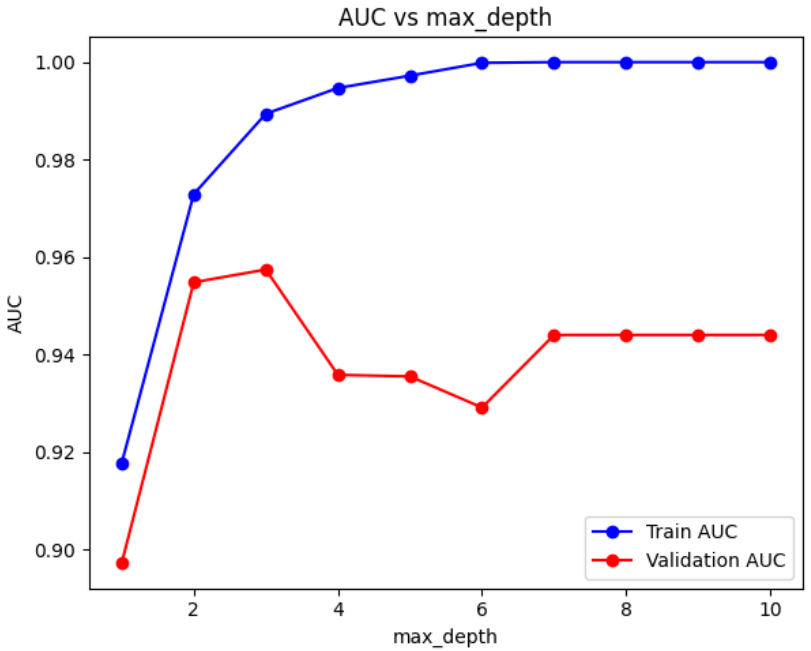
- criterion is the function to measure the quality of a split
- splitter is the strategy used to choose the split at each node
- max_depth is the maximum depth of the tree
- min_samples_split is the minimum number of samples required to split an internal node
- min_samples_leaf is the minimum number of samples required to be at a leaf node

5. Models

Decision tree classifier

- Results on the breast cancer Wisconsin classification dataset (AUC=Area Under the Curve, higher is better)
- The learning capacity of the model grows with max_depth
- The model underfits for max_depth<3, overfits for max_depth>3, and attains an optimal balance at max_depth=3





5. Models

Random forest classifier

- A random forest is a classifier that fits a number of decision tree classifiers on various sub-samples of the training set
- It uses averaging to improve the predictive accuracy and control over-fitting
- This classifier has native support for missing values (NaNs)
- Most configuration options are passed on to the underlying instances of the DecisionTreeClassifier class

5. Models

Random forest classifier

- Random forest classifier:

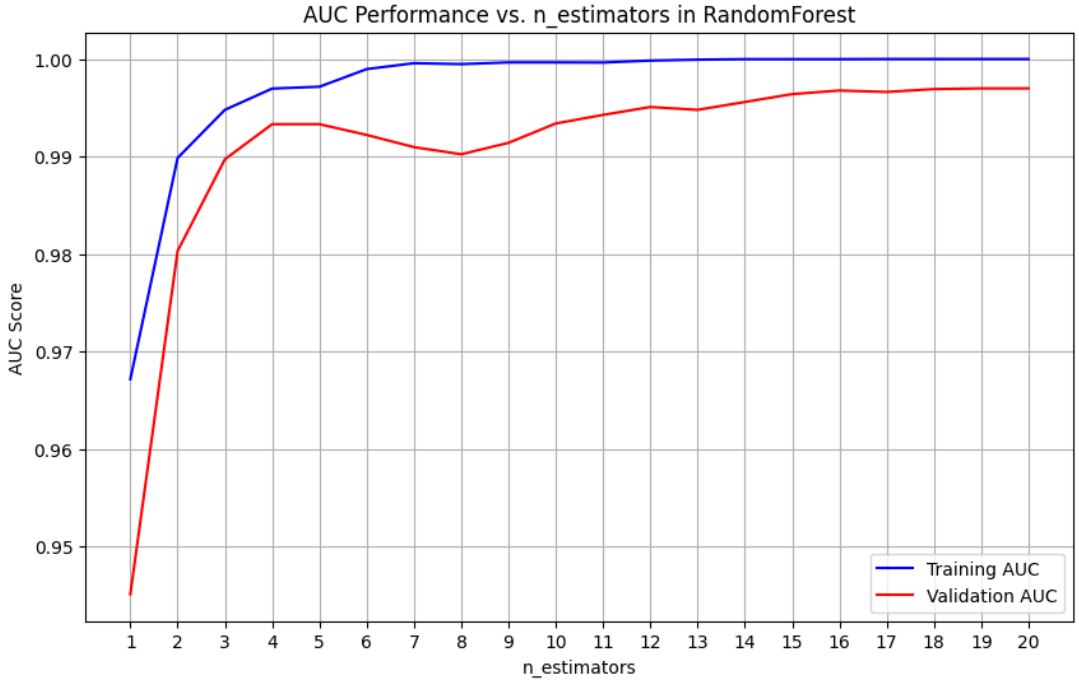
sklearn.ensemble.RandomForestClassifier(n_estimators, criterion, max_depth, min_samples_split, min_samples_leaf)

- n_estimators is the number of trees in the forest
- criterion is the function to measure the quality of a split
- max_depth is the maximum depth of the tree
- min_samples_split is the minimum number of samples required to split an internal node
- min_samples_leaf is the minimum number of samples required to be at a leaf node

5. Models

Random forest classifier

- Results on the breast cancer Wisconsin classification dataset (AUC=Area Under the Curve, higher is better)
- The performance enhances steadily with n_estimators until a stable point is reached
- More decision trees do not increase performance beyond that point



5. Models
k-means clustering

sklearn.cluster.KMeans(n_clusters, init, n_init, max_iter)

- n_clusters is the number of clusters to form as well as the number of centroids to generate
- init is the method for initialization
- n_init is the number of times the k-means algorithm is run with different centroid seeds
- max_iter is the maximum number of iterations

6. CONCLUSION

6. Conclusion

- The sklearn package provides reliable implementations for a wide range of standard machine learning models for classification, regression and clustering
- The features matrix and the label array must belong to standard classes of the numpy, pandas or scipy packages
- The interface of the model classes is standardized
- Utilities for cross validation, hyperparameter optimization, and performance measurement are provided

Bibliography

- The official course book is: Russell, S. & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th edition). Pearson.
- Sections of the book related to this course:
  - Naïve Bayes: section 12.6
  - Decision trees: section 19.3
  - Cross-validation: section 19.4
  - Linear regression: section 19.6
  - Logistic regression: section 19.6
  - Support Vector Machines: section 19.7
  - Random forests: section 19.8
  - Multilayer perceptrons: section 21.1
- You may also see Chapters 24 and 26 of this book: Guttag, John V. (2021). Introduction to computation and programming using Python with application to understanding data. MIT Press. 3rd edition.